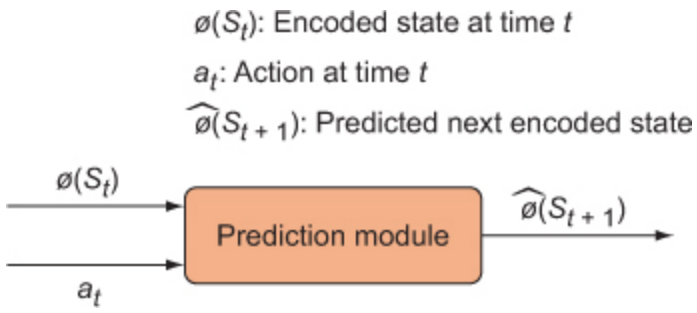
The encoder model is trained via the inverse model because we actually use the encoded states as inputs to the forward and inverse models *f* and *g* rather than the raw states. That is, the forward model becomes a function, $f: \phi(S_t) \times a_t \rightarrow \hat{\phi}(S_{t+1})$, where $\hat{\phi}(S_{t+1})$ refers to a prediction of the encoded state, and the inverse model becomes a function, $g: \phi(S_t) \times \hat{\phi}(S_{t+1}) \rightarrow \hat{a}_t$ (figure 8.9). The notation $P: a \times b \rightarrow c$ means that we define some function *P* that takes a pair (*a*,*b*) and transforms it into a new object *c*.

Figure 8.9. The forward-prediction module actually uses encoded states, not the raw states. Encoded states are denoted φ(S_t) or $\tilde{S}_t$.



The encoder model isn't trained directly—it is *not* an autoencoder. It is only trained through the inverse model. The inverse model is trying to predict the action that was taken to transition from one state to the next using the encoded states as inputs, and in order to minimize its own prediction error, its error will backpropagate through to the encoder model as well as itself. The encoder model will then learn to encode states in a way that is useful for the task of the inverse model. Importantly, although the forward model also uses the encoded states as inputs, we do *not* backpropagate from the forward model to the encoder model. If we did, the forward model would coerce the encoder model into mapping all states into a single fixed output, since that would be the easiest to predict.

Figure 8.10 shows the overall graph structure: the forward pass of the components and also the backward (backpropagation) pass to update the model parameters. It is worth repeating that the inverse model